

```

✳ = DiVRoC().apply # set up splatting
sigmas = torch.linspace(1,5,3) # define smoothing scales
# set optimisable value parameters
params = torch.nn.Parameter(torch.zeros(1, C, K, 1, 1))
# differentiable OFF-GRID coordinate 3D points
pts = ctrl_pts.reshape(1, -1, 1, 1, 3)
K,C = params.shape[:2] # K = num_pts, C = num_channels
cp_weights = torch.nn.Parameter(torch.ones(1, K, len(sigmas))
\
        /len(sigmas)) # uniform weights for each
        scale
# DEFINE OPTIMISER AND TRAINING LOOP
# keep weights positive and unit sum
cp_soft = torch.softmax(cp_weights, dim=-1)
splat_ones = ✳(torch.ones_like(pts[...,:1]).transpose(2,
1),\
        pts, (1, 1, H, W, D)) # make values homogeneous
splat_values = ✳(torch.tanh(params, pts, (1, C, H, W, D))
splat_cp_w = ✳(cp_soft.transpose(2, 1)[...,None,None], pts,\
        (1, len(sigmas), H, W, D))
# MULTI-SIGMA SMOOTHING
cumulative_weights = torch.zeros(1,1,H,W,D)
cumulative_values = torch.zeros(1,C,H,W,D)
for i_, sigma in enumerate(sigmas):
    σ = GaussianSmoothing(sigma)
    # Smooth values and density
    ones_smooth = σ(splat_ones)
    values_smooth = σ(splat_values)
    # Smooth the specific scale weights
    weights_smooth =
σ(splat_cp_w[:,i_:i_+1,...]).div(ones_smooth+1e-6)
    cumulative_weights += weights_smooth
    cumulative_values +=
values_smooth.div(ones_smooth+1e-6)*weights_smooth
🚀 = (cumulative_values/cumulative_weights.add(1e-6))

```